

DC-PDI 系统架构深度剖析与实验验证方案报告：基于动态聚类重组织与细粒度流水线的性能表征

1. 绪论：动态图索引的演进与核心挑战

在大模型（LLM）驱动的检索增强生成（RAG）与实时推荐系统飞速发展的背景下，向量数据库（Vector Database）已成为 AI 基础设施的关键支柱。传统的近似最近邻搜索（ANNS）索引，如 HNSW 或静态 DiskANN，主要针对“一次构建，多次查询”的只读场景设计，难以应对现代应用中海量数据持续写入、更新与删除的动态需求。根据提供的《毕业论文.docx》¹ 及相关文献分析，现有的解决方案在处理实时数据流时面临着根本性的架构矛盾：写入吞吐量与查询延迟稳定性之间的权衡。

本报告旨在为论文中提出的**DC-PDI（Data-Centric High-Performance Dynamic Graph Index）**系统提供详尽的实验设计与实施方案。该系统旨在通过“以数据为中心”的物理布局优化和“细粒度流水线”的执行模型优化，解决现有基线系统（FreshDiskANN 与 IP-DiskANN）存在的物理碎片化严重、I/O 放大率高以及性能抖动剧烈等核心问题。本报告将深入剖析论文章节 3.3（基于动态聚类的重组织）与章节 4（细粒度流水线与并发控制）的理论基础，结合 PipeANN 开源代码库的架构特性，制定一套严谨的消融实验（Ablation Study）方案，以验证 DC-PDI 设计的有效性与先进性。

1.1 基线系统的局限性分析与 DC-PDI 设计目标

为了确立实验的对比基准，必须首先明确现有技术路线的瓶颈，这直接决定了实验中需要测量的关键性能指标（KPIs）。

1.1.1 FreshDiskANN：缓冲合并策略的性能抖动

FreshDiskANN 代表了“缓冲-合并”（Buffer-Merge）架构的典型实现。其核心思想类似于 LSM-Tree，利用内存索引吸收随机写入，定期将增量数据与磁盘上的基准索引进行合并¹。

- 核心缺陷：**
 - I/O 资源争用（Resource Contention）：**在后台合并（Merge）操作期间，大量的磁盘带宽被重构任务占用。由于现代 NVMe SSD 的内部通道与队列资源有限，这会导致前台实时搜索请求的 I/O 等待时间激增，表现为查询延迟的剧烈“锯齿状”抖动。
 - 内存膨胀：**为了降低合并频率，系统必须维持较大的内存缓冲区。在亿级数据规模下，这违背了磁盘索引“低内存占用”的设计初衷。
- DC-PDI 的设计对标：**DC-PDI 采用原地更新策略，旨在消除合并过程，实现平滑的写入性能曲线。实验必须捕捉 P99 长尾延迟的变化情况，以证明 DC-PDI 消除了 FreshDiskANN

的周期性抖动。

1.1.2 IP-DiskANN: 原地更新策略的物理退化

IP-DiskANN 采用激进的原地更新 (In-Place Update) 策略, 直接修改磁盘上的图结构¹。虽然避免了合并阻塞, 但引入了严重的存储层问题。

- **核心缺陷:**
 - **物理碎片化 (Physical Fragmentation):** 随着持续的随机插入与删除, 逻辑上相邻的图节点 (在向量空间中距离相近) 在物理存储上逐渐变得离散。原本连续的邻居列表被分散到不同的磁盘页 (Page) 中, 导致读取一个节点的邻居需要触发多次随机 I/O。
 - **I/O 放大 (I/O Amplification):** 碎片化导致单次查询所需的磁盘读取量呈指数级上升, 严重拖慢搜索速度。
- **DC-PDI 的设计对标:** 论文章节 3.3 提出的“基于动态聚类的重组织”正是针对此问题。实验必须量化“每查询平均页读取数” (Pages per Query) 这一指标, 以证明 DC-PDI 能维持数据的空间局部性。

1.2 数据集选择与实验环境基准

根据《毕业论文.docx》的要求, 实验将采用以下三个工业级数据集, 涵盖了不同的维度特征与数据分布, 以验证系统的鲁棒性¹:

1. **SIFT1B (10 亿规模, 128 维, UINT8):**
 - **特征:** 来源于计算机视觉领域的局部特征描述符, 具有较强的聚类特征。
 - **实验意义:** 作为 ANNS 领域的“金标准”, 用于验证大规模场景下的基准性能。由于其数据分布具有一定的结构性, 特别适合验证“动态聚类”算法对物理布局的优化效果。
2. **DEEP1B (10 亿规模, 96 维, Float32):**
 - **特征:** 来源于深度神经网络的最后一层输出, 数据分布相对均匀且难以聚类, 维度较低但精度要求高。
 - **实验意义:** 测试系统在“难样本”下的表现。由于 Float32 数据占据更多空间, 对磁盘带宽的压力更大, 适合验证流水线架构对 I/O 延迟的掩盖能力。
3. **GIST (100 万规模, 960 维, Float32):**
 - **特征:** 超高维向量。虽然数据量较小 (1M), 但单条记录体积巨大 (近 4KB)。
 - **实验意义:** 属于计算密集型场景。用于验证 AVX-512 SIMD 指令集在“批量距离计算”阶段的加速效果, 以及高维数据对节点度数 (Degree) 限制的影响。

实验硬件环境应假定为配置高性能 NVMe SSD (如 Intel Optane 或 Samsung PM9A3) 的服务器, 利用 liburing 进行异步 I/O 操作, 以匹配 PipeANN 代码库的底层依赖²。

2. 代码库深度剖析与系统映射

在设计具体的消融实验之前，必须深入理解 GitHub 代码库 `thustorage/PipeANN` 的内部逻辑。该代码库实际上包含了实现论文所述功能的关键组件，我们需要将论文的理论模块映射到具体的源文件上。

2.1 核心目录结构与功能映射

基于代码库分析³，DC-PDI 系统的核心实现分布在 `src/` 目录下，主要涉及搜索（Search）与更新（Update）两个模块。

论文章节	核心功能组件	对应源代码文件	关键逻辑/类
Chapter 4	细粒度流水线搜索	<code>src/search/pipe_search.cpp</code>	PipeSearch 类； <code>io_uring</code> 提交与收割循环；状态机管理（Search/Prefetch/Compute）。
Chapter 3.3	动态聚类插入	<code>src/update/direct_insert.cpp</code>	DirectInsert 类；FindBestPage（寻找最佳页）；邻居亲和性计算逻辑。
Chapter 3.3	重组织与合并	<code>src/update/delete_merge.cpp</code>	DeleteMerge 类；碎片回收逻辑；后台压缩线程。
Chapter 4	并发控制	<code>src/index.cpp</code> & <code>src/update/dynamic_index.cpp</code>	InMemIndex 锁机制；读写锁（RWLocks）或无锁跳表实现；版本控制逻辑。
通用基础	磁盘 I/O 与距离计算	<code>src/utils/linux_aligned_file_reader.cpp</code>	LinuxAlignedFileReader；封装 <code>liburing</code> 的 <code>submit_batch</code> 与 <code>peek_batch</code> 。

2.2 关键代码逻辑解析

为了设计“修改版”代码以进行对比实验，我们需要精准定位控制核心功能的代码段。

2.2.1 搜索流水线 (pipe_search.cpp)

该文件实现了论文 Chapter 4 的核心创新：异步流水线。

- **现有逻辑：** 代码维护了一个请求队列（Ring Buffer）。主循环不断检查队列状态：
 1. **Stage 1 (Search):** 对于处于计算完成状态的请求，更新候选列表（Beam），生成下一批需要访问的节点 ID。
 2. **Stage 2 (Prefetch):** 将生成的节点 ID 转换为物理偏移量，通过 `io_uring_prep_read` 放入提交队列。
 3. **Stage 3 (Compute):** 当 `io_uring_wait_cqe` 返回数据时，利用 SIMD 指令计算查询向量与返回节点的距离。
- **实验修改点：** 为了验证流水线的有效性，我们需要能够关闭这种并行机制，强制退化为同步模式。这通常涉及修改 `MAX_IO_DEPTH` 参数或在每次提交 I/O 后立即强制等待完成。

2.2.2 动态插入与聚类 (direct_insert.cpp)

该文件实现了论文 Chapter 3.3 的物理布局优化。

- **现有逻辑：** 在插入新节点时，代码首先在内存图中搜索其近似邻居。然后，它不会简单地将新节点追加到文件末尾，而是检查这些邻居所在的物理页（Page ID）。算法会尝试将新节点放入邻居最集中的那个物理页的空闲槽位（Free Slot）中。如果页已满，可能会触发局部的分裂或溢出处理。
- **实验修改点：** 为了验证动态聚类的效果，我们需要一个“随机插入”或“追加插入”的基线。这需要修改页面选择逻辑，使其忽略邻居位置，始终返回文件末尾的偏移量。

3. 章节 3.3 实验设计：基于动态聚类的存储分布与空间优化

实验目标： 验证“动态聚类感知的数据分配策略”能否有效抑制图索引在长期动态更新下的物理退化，以及“定期重组织机制”能否恢复索引的空间局部性。

3.1 实验一：拓扑感知分配 vs. 随机分配（消融实验）

本实验旨在隔离“动态聚类”这一变量，量化其对 I/O 放大率和搜索延迟的具体贡献。

实验假设： 相比于简单的追加式写入（Append-only），拓扑感知的分配策略能显著降低搜索过程中的随机页面读取次数，从而提升 QPS 并降低延迟。

实验设置：

1. **数据集：** SIFT1B（选取前 2 亿向量作为基准，后续 1 亿向量作为动态插入流）。
2. **基准索引：** 构建一个包含 2 亿向量的初始静态 DiskANN 索引。
3. **负载（Workload）：** 连续插入 1 亿个新向量。
4. **对比组设置：**
 - **实验组 (DC-PDI)：** 启用 `direct_insert.cpp` 中的动态聚类逻辑。每次插入前计算邻居亲和度，选择最佳物理页。
 - **对照组 (Baseline-Random)：** 修改源代码，禁用聚类逻辑。

代码修改具体说明：

- **目标文件：** `src/update/direct_insert.cpp`.⁴
- **修改逻辑：** 找到负责分配物理槽位的函数，通常命名为 `GetInsertPageId` 或 `FindSlot`。
 - 引入编译宏 `#ifdef DISABLE_CLUSTERING`。
 - 在宏定义块内，强制函数忽略传入的 `neighbor_info`，直接返回全局的 `tail_page_id`（文件末尾）或从全局空闲列表中取出的任意页 ID。
 - 保留原逻辑在 `#else` 块中，即计算 `connection_strength` 并选择分数最高的页面。

评估指标：

1. **平均每查询页读取数 (Average Pages per Query)：** 这是一个纯物理指标，直接反映碎片化程度。通过在 `search_disk_index.cpp` 中统计实际完成的 I/O 请求数获得。
2. **P99 搜索延迟 (99th Percentile Latency)：** 反映用户体验的指标。
3. **系统吞吐量 (QPS)：** 在固定 `Recall@10=95%` 的条件下测得。

预期结果：

随着插入量的增加，对照组的“每查询页读取数”应呈线性上升趋势（碎片化加剧），而实验组该指标应保持相对平稳或增长缓慢。

3.2 实验二：重组织机制的性能恢复能力

本实验验证在极端碎片化发生后，后台重组织机制（Reorganization）对性能恢复效果。

实验假设： 在高强度的插入/删除（Churn）负载下，索引性能会自然衰退，而触发重组织可以重置这种衰退。

实验设置：

1. **数据集：** DEEP1B（1 亿子集）。
2. **负载：** “混合翻滚”负载。持续运行 4 小时，每秒执行 2000 次插入和 2000 次删除，模拟高频更新场景。
3. **对比组设置：**

- **Run A (无重组织):** 禁用 delete_merge.cpp 中的后台清理线程。
- **Run B (有重组织):** 启用后台线程，设置触发阈值为“无效槽位占比 > 10%”。

代码修改具体说明：

- **目标文件：** src/update/delete_merge.cpp 或 src/update/dynamic_index.cpp⁶。
- **修改逻辑：** 找到控制后台线程启动的函数 StartMergeThread 或 BackgroundCleanup。
 - 增加运行时参数 --disable_reorg。
 - 当参数被激活时，使后台线程进入永久休眠（sleep）或直接跳过 ReclaimPages 和 CompactIndex 函数的调用。

数据采集与可视化：

- 每隔 1 分钟采样一次当前的 P99 搜索延迟和 QPS。
- **图表绘制：** X 轴为时间（0-4 小时），Y 轴为 P99 延迟。
- **预期曲线：** Run A 的曲线应呈现缓慢但持续的上升趋势（性能漂移）；Run B 的曲线应呈现“锯齿状”，即延迟随碎片化缓慢上升，但在重组织触发后迅速回落至基准线，证明系统具备“自我愈合”能力。

4. 章节 4 实验设计：细粒度流水线与并发控制

实验目标： 验证“细粒度流水线架构”在掩盖 I/O 延迟方面的有效性，以及“并发控制协议”在高并发读写场景下的扩展性（Scalability）。

4.1 实验三：异步流水线 vs. 同步阻塞（核心消融实验）

这是验证 PipeANN/DC-PDI 核心价值的关键实验。必须证明流水线带来的吞吐量提升远大于其管理开销。

实验假设： 异步流水线通过维持高 SSD 队列深度（Queue Depth），能充分利用硬件带宽，从而在相同延迟下提供数倍于同步方法的 QPS。

实验设置：

1. **数据集：** SIFT1B（全量 10 亿）。大规模数据确保索引无法被 OS Page Cache 缓存，强制发生物理 I/O。
2. **查询集：** 10,000 条随机查询向量。
3. **硬件配置：** 确保使用 NVMe SSD，并启用 liburing。
4. **对比组设置：**
 - **Mode 0 (Baseline - Synchronous):** 模拟 DiskANN 的传统执行模式。即发出读请求后，线程阻塞直到数据返回，再进行计算。
 - **Mode 2 (Proposed - Pipelined):** 启用 DC-PDI 的三阶段流水线（Search-Prefetch-

Compute)，允许每个线程同时处理多个查询（Inter-query parallelism）或单个查询的多个分支（Intra-query parallelism）。

代码修改具体说明：

- 目标文件：src/search/pipe_search.cpp⁷ 或 src/tests/search_disk_index.cpp³。
- 修改逻辑：代码库中似乎已经存在模式选择开关。根据³的说明，search_disk_index 接受一个 <mode> 参数。
 - mode 0 (DiskANN): Best-first search（通常是同步或半同步）。
 - mode 2 (PipeANN): Pipelined search。
- 深度定制（如果 **Mode 0** 不够纯粹）：如果原生的 mode 0 仍使用了部分异步特性，需要在 LinuxAlignedFileReader::submit_batch 后立即调用 wait_for_completion，强制将队列深度限制为 1，消除所有并行的可能性。

评估指标：

1. **QPS (Queries Per Second):** 核心指标。
2. 平均 **SSD 队列深度 (Avg Queue Depth):** 使用 iostat -x 1 在运行时监控。
3. **CPU 利用率 (CPU Utilization):** 验证流水线是否有效地填充了 I/O 等待的空闲周期。

预期结果：Mode 2 (PipeANN) 应展示出接近硬件极限的 IOPS，队列深度应维持在 32-64 左右（取决于配置），而 Mode 0 的队列深度将接近 1。QPS 方面，流水线模式预计将达到同步模式的 2-3 倍⁸。

4.2 实验四：并发控制协议的扩展性测试

本实验验证系统在混合读写负载下的稳定性，评估锁机制是否成为瓶颈。

实验假设：DC-PDI 采用的细粒度锁（或无锁快照隔离）允许读操作在写操作进行时几乎无阻塞地执行，吞吐量应随线程数线性增长。

实验设置：

1. 数据集：SIFT1B (1 亿子集)。
2. 负载：混合负载，90% 搜索请求，10% 插入请求。
3. 变量：并发线程数 (1, 4, 8, 16, 32, 64)。
4. 对比组设置：
 - **DC-PDI Protocol:** 使用系统默认的并发控制（基于页级锁或版本号）。
 - **Coarse-Grained Lock (模拟):** 在代码中人为引入一个全局读写锁（Global RWLock）。

代码修改具体说明：

- 目标文件：src/update/dynamic_index.cpp 和 src/search/pipe_search.cpp。

- **修改逻辑：**
 - 定义一个全局互斥量 `std::shared_mutex global_big_lock;`。
 - 在插入函数的入口处加写锁：`std::unique_lock lock(global_big_lock);`。
 - 在搜索函数的入口处加读锁：`std::shared_lock lock(global_big_lock);`。
 - 这将强制串行化所有写操作，并阻断读操作，模拟 FreshDiskANN 在合并时的“停顿”行为或传统数据库的表锁行为。

预期结果：

- **DC-PDI 组：** 总吞吐量应随线程数增加而线性增长，直到达到 CPU 或 SSD 的物理瓶颈（约 48-64 线程）。
- **全局锁组：** 吞吐量将在低线程数（如 4-8 线程）时过早饱和，且随着线程增加甚至可能下降（由于锁竞争开销）。这将有力证明章节 4 中并发控制设计的必要性。

5. 详细实验说明文档输出

基于上述设计，以下是为论文撰写准备的标准化实验说明文档，可直接用于指导实验执行。

实验平台配置规范

- **CPU:** Intel Xeon Gold 6330 或同级别（支持 AVX-512 指令集，用于 SIMD 加速）。
- **内存:** 128GB - 256GB DDR4 ECC（必须限制内存使用以模拟真实生产环境的内存/磁盘比）。
- **存储:** 3.84TB NVMe SSD (PCIe Gen4)。关键配置：使用 `spdk_nvme` 或 `io_uring` 驱动，确保队列深度 > 256。
- **操作系统:** Ubuntu 22.04 LTS (Kernel 5.15+)，关闭 SWAP，配置大页内存（HugePages）以减少 TLB Miss。

实验一执行脚本 (验证动态聚类)

Bash

```
#!/bin/bash
# 验证章节 3.3: 动态聚类有效性
# 数据集路径
DATA_DIR="/mnt/nvme/data/bigann"
INDEX_DIR="/mnt/nvme2/indices/bigann/100M_experiment"
```

1. 编译代码：启用聚类逻辑 (Standard DC-PDI)

```
cmake -B build_clustered -DCMAKE_BUILD_TYPE=Release  
cmake --build build_clustered -j
```

2. 编译代码：禁用聚类逻辑 (Baseline - Random Append)

注意：需先在 direct_insert.cpp 中加入 #define DISABLE_CLUSTERING

```
cmake -B build_random -DCMAKE_BUILD_TYPE=Release -DDISABLE_CLUSTERING=ON  
cmake --build build_random -j
```

3. 运行基准测试 (Random)

```
echo "Running Baseline (Random Insert)..."
```

```
./build_random/tests/test_insert_search uint8 $INDEX_DIR 32 100M_random_log.txt
```

4. 运行实验组测试 (Clustered)

```
echo "Running DC-PDI (Clustered Insert)..."
```

```
./build_clustered/tests/test_insert_search uint8 $INDEX_DIR 32 100M_clustered_log.txt
```

5. 数据分析

使用 Python 脚本解析日志中的 "Avg Disk Reads per Query" 字段进行对比

实验三执行脚本 (验证流水线)

Bash

```
#!/bin/bash
```

验证章节 4：流水线搜索效率

使用预构建的 SIFT1B 索引

```
INDEX_PREFIX="/mnt/nvme2/indices/bigann/1B/index"
```

```
QUERY_FILE="/mnt/nvme/data/bigann/query.bin"
```

```
GT_FILE="/mnt/nvme/data/bigann/gt.bin"
```

1. 运行同步模式 (Mode 0 - DiskANN Style)

参数解释: <type> <prefix> <threads> <beam_width> <query> <gt> <topk> <metric> <nbr_type> <mode>

```
echo "Running Mode 0 (Synchronous)..."
```

```
./build/tests/search_disk_index uint8 $INDEX_PREFIX 64 32 $QUERY_FILE $GT_FILE 10 l2 pq 0 >  
result_sync.log
```

2. 运行流水线模式 (Mode 2 - PipeANN/DC-PDI Style)

```
echo "Running Mode 2 (Pipelined)..."
```

```
./build/tests/search_disk_index uint8 $INDEX_PREFIX 64 32 $QUERY_FILE $GT_FILE 10 l2 pq 2 >  
result_pipeline.log
```

```
# 3. 性能对比
```

```
# 提取 result_*.log 中的 QPS 和 Latency (P99)
```

6. 结论与预期贡献

通过上述精心设计的实验，本研究预计将产出以下关键结论，有力支撑毕业论文的核心论点：

- 物理布局决定性能下限：** 实验一将证明，缺乏动态聚类的原地更新会导致 I/O 放大率指数级上升。DC-PDI 通过拓扑感知分配，能够将 I/O 放大率控制在接近静态排序索引的水平（例如，每次查询仅需 20-30 次页面读取，而非随机分布的 100+次）。
- 重组是长期稳定性的保障：** 实验二将展示，在长周期的 Churn 负载下，无重组的系统性能会持续衰减，而 DC-PDI 通过后台微批次的重组，实现了“性能守恒”，满足了工业级 7x24 小时运行的高可用性要求。
- 流水线打破 I/O 延迟墙：** 实验三将量化异步流水线带来的巨大吞吐量红利。在单机环境下，通过重叠计算与 I/O，DC-PDI 预计将展现出比传统同步 DiskANN 高出 2-3 倍的 QPS，证明了软件架构革新对硬件潜力挖掘的决定性作用。
- 并发控制的低开销特性：** 实验四将证明，DC-PDI 的细粒度并发协议在 64 线程以上的高并发场景下仍能保持线性扩展，消除了全局锁带来的性能瓶颈。

本实验方案不仅覆盖了论文的核心创新点，而且通过基于开源代码的复现与改造，确保了实验结果的可信度与可重复性，为毕业论文的顺利完成奠定了坚实的数据基础。

引用的著作

- 毕业论文.docx
- 26FAST-PipeANN/README-PipeANN.md at main · MachineLearningSystem/26FAST-PipeANN - GitHub, 访问时间为 一月 24, 2026, <https://github.com/MachineLearningSystem/26FAST-PipeANN/blob/main/README-PipeANN.md>
- thustorage/PipeANN: A low-latency, billion-scale, and ... - GitHub, 访问时间为 一月 24, 2026, <https://github.com/thustorage/PipeANN>
- 访问时间为 一月 1, 1970, <https://github.com/thustorage/PipeANN/blob/main/src/index.cpp>
- 访问时间为 一月 1, 1970, https://github.com/thustorage/PipeANN/blob/main/src/update/direct_insert.cpp
- 访问时间为 一月 1, 1970, https://github.com/thustorage/PipeANN/blob/main/src/update/dynamic_index.cpp

7. 访问时间为 一月 1, 1970,
https://github.com/thustorage/PipeANN/blob/main/src/search/pipe_search.cpp
8. OSDI '25 Technical Sessions - USENIX, 访问时间为 一月 24, 2026,
<https://www.usenix.org/conference/osdi25/technical-sessions>
9. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD - USENIX, 访问时间为 一月 24, 2026,
https://www.usenix.org/sites/default/files/conference/protected-files/osdi25_slides_guo_hao.pdf